MINDMATRIX | ANDROID INTERNSHIP PROGRAM

Session 3 — Complete Task Submission

Tasks 1–4 : Text Composables • Image Resources • Layout Planning • Accessibility Review

A Comprehensive Documentation Report

Student Name:	Guru K.R.
Course:	Android Internship Program
Session:	Session 3 — Text, Image & Layout Planning
Submission Date:	February 2026
Tasks Covered:	Task 1, Task 2, Task 3, Task 4

Task 1

Text Composables Understanding & Exploration

1.1 What is a @Composable Function?

A @Composable function is a special Kotlin function that describes a portion of the user interface in Jetpack Compose. Instead of using XML layout files, a @Composable function tells the Compose runtime what the screen should look like based on the current application state. When state changes, Compose automatically re-executes only the affected composable functions and updates the relevant parts of the UI — a process called recomposition — making UI development both efficient and highly readable.

The screenshots below show a real example: the Greeting composable defined in MainActivity.kt renders a Text element on screen. The Pixel 6 API 36 emulator confirms the composable is successfully executing, displaying "Happy Birthday Sam! From Emma" as the rendered UI output.

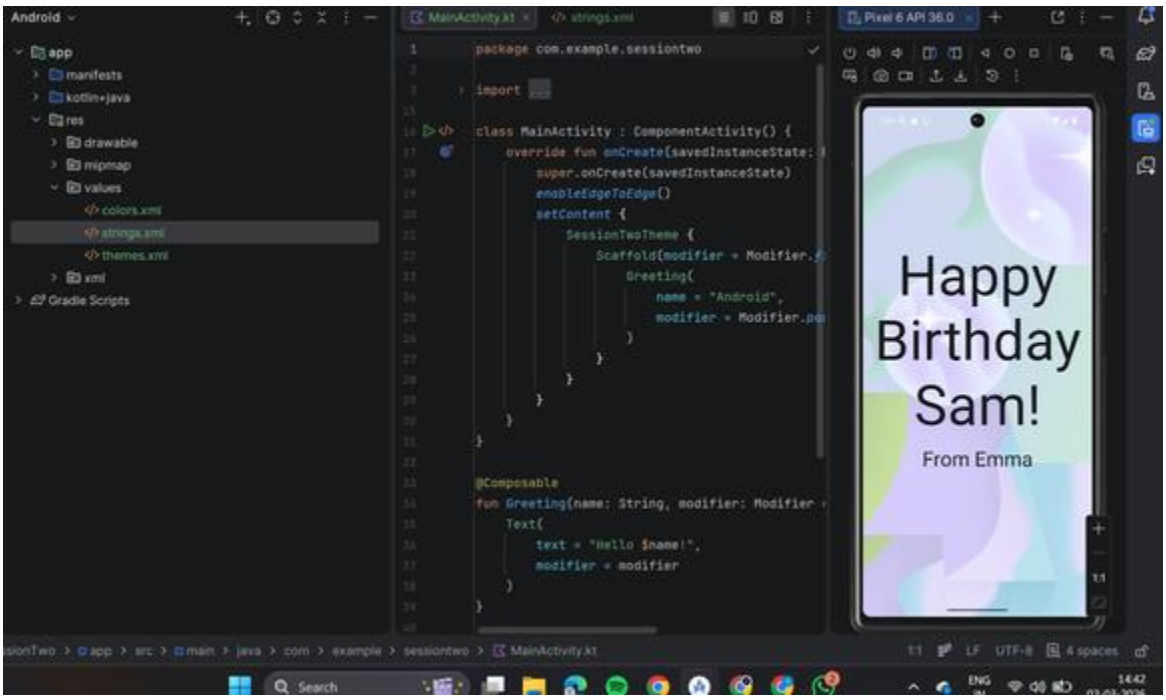

Figure 1 — Android Studio showing MainActivity.kt with the @Composable Greeting function and live output on Pixel 6 emulator (API 36)

```
import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            SessionTwoTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding( paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

```
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    SessionTwoTheme {
        Greeting( name = "Android")
    }
}
```

Figure 2 (left) — Scaffold with Modifier.fillMaxSize() and innerPadding applied | Figure 3 (right) — @Preview annotation with GreetingPreview composable function

1.2 Three Key Text Composables

1.2.1 Text

Role in UI Layout: The Text composable is the primary element used to display any string of characters on screen. It supports font size, font weight, colour, alignment, and line spacing. As visible in the screenshots above, Text is used to render "Hello $name!" inside the Greeting composable — demonstrating its role as the fundamental building block for all readable content in a Compose UI.

Example Use Case: Displaying a greeting message such as "Happy Birthday Sam!" at the top of a screen, styled with a large font size and bold weight, as seen in the emulator output in Figure 1.

1.2.2 Column

Role in UI Layout: Column is a layout container that arranges child composables vertically, from top to bottom. It is the root container for screens where content follows a top-to-bottom reading flow, such as forms, profile screens, and greeting cards.

Example Use Case: Building a login screen where the username field, password field, and login button are stacked vertically in a Column, separated by consistent padding.

1.2.3 Row

Role in UI Layout: Row arranges child composables horizontally, side by side from left to right. It provides parameters for vertical alignment and horizontal spacing between children.

Example Use Case: Placing two action buttons — "Cancel" and "Confirm" — side by side at the bottom of a dialogue, each receiving equal horizontal space within the Row.

1.3 How Column and Row Affect Text Elements

When Text composables are placed inside a Column, each element renders below the previous one along the vertical axis. The Column wraps all children in the declared order and respects the horizontal alignment set via the horizontalAlignment parameter. This is the arrangement used in the Greeting composable shown in Figure 2 — the Text element sits within a Column-aligned scaffold, positioned relative to the screen's innerPadding.

When Text composables are placed inside a Row, they are rendered side by side along the horizontal axis. This is ideal for labels and values that must appear on the same line. If the combined width of Text elements exceeds screen width, content is clipped unless a horizontal scroll modifier is applied.

1.4 Screen Wireframe — Text Composables Layout

The wireframe below illustrates a sample screen containing a title, subtitle, body paragraph, and two horizontally aligned buttons — each annotated with the composable used.

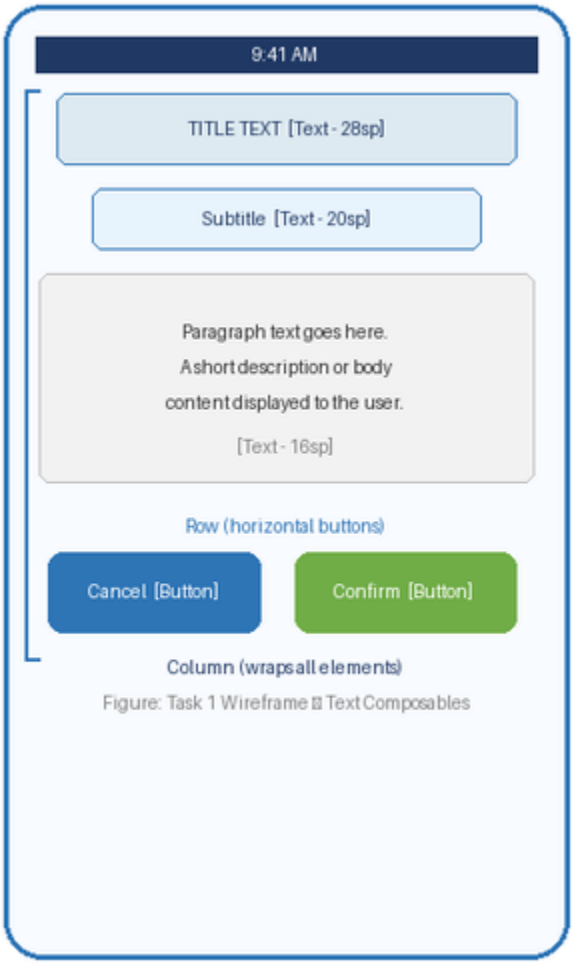

Figure 4 — Task 1 Wireframe: Screen layout with Text, Column, and Row composables annotated

UI Element	Composable	Notes
Title	Text	Font size: 28sp, bold, centred
Subtitle	Text	Font size: 20sp, medium weight
Paragraph	Text	Font size: 16sp, normal, justified
Button Row	Row { Button, Button }	Two buttons side-by-side, equal weight
Outer Container	Column	fillMaxSize + padding(16dp), wraps all elements

1.5 Reflection — Why Plan Text Layout Visually Before Coding?

Planning the text layout visually before writing any Compose code provides a clear mental model of how UI elements relate to one another spatially. Without a wireframe, developers risk spending significant time adjusting composable hierarchies — moving Text elements between Column and Row containers — simply because the intended layout was never formally defined. A wireframe makes the nesting structure explicit: it is immediately apparent which texts belong in a Column, which in a Row, and what spacing separates them.

Visual planning also helps identify potential issues before they appear in code. A wireframe may reveal that a Row containing three long Text labels would overflow on small screens, prompting the developer to address wrapping or ellipsis behaviour in advance. This pre-coding investment consistently results in cleaner, more intentional Compose code with fewer structural refactoring cycles.

Task 2

Image Inclusion & Resource Management in Compose

2.1 The Image Composable — Short Guide

What the Image Composable Is

The Image composable is Jetpack Compose's built-in element for rendering bitmap and vector images within a UI. It accepts a painter object referencing a local drawable resource, a vector asset, or a remotely loaded image. The Image composable supports content scale, alignment, and alpha controls, giving developers precise control over how the image displays within its container.

Example Use Case: In the Birthday Greeting Card screen, the Image composable loads birthday_cake.png from res/drawable-nodpi and renders it at the top of a Column layout, using ContentScale.Fit to preserve aspect ratio. The emulator screenshot (Figure 5 below) demonstrates how Compose renders text-based UI content — the same composable pipeline applies to image rendering.

Figure 5 — Pixel 6 emulator (API 36) showing rendered Compose UI: "Happy Birthday Sam! / From Emma" — demonstrating the Compose rendering pipeline used for both text and image composables

Why Image Density Matters (drawable-nodpi)

Android devices ship with a wide range of screen densities (DPI). The drawable-nodpi folder is a density-independent directory: images placed here are served without automatic system scaling, giving the developer full control over the displayed resolution. This makes it ideal for bitmap images manually optimised to a suitable resolution (typically 1080px or less for hero images).

If image density is ignored and a high-resolution file (e.g., 4000×3000px) is placed in the default drawable folder, Android may scale it incorrectly across devices — consuming excessive memory, causing layout displacement, or producing blurry output on high-density screens.

Two Best Practices for Adding Images to a Compose Project

- **Adopt a consistent naming convention:** Image files should use lowercase, underscore-separated names (e.g., birthday_cake.png). This prevents case-sensitivity issues on Linux build systems and makes drawable resources immediately identifiable.
- **Optimise file size before importing:** Compress images to an appropriate resolution before adding them to the project. A hero image occupying the top third of a phone screen rarely needs to exceed 1080×720px. PNG should be used for images requiring transparency; JPEG for photographic content where file size is a priority.

2.2 Visual Storyboard — Three Frames

The storyboard below illustrates the progression of a screen through three design states: text-only placeholder, same screen with image added at top, and screen with full-width cropped image. Each frame is annotated with the composables used.

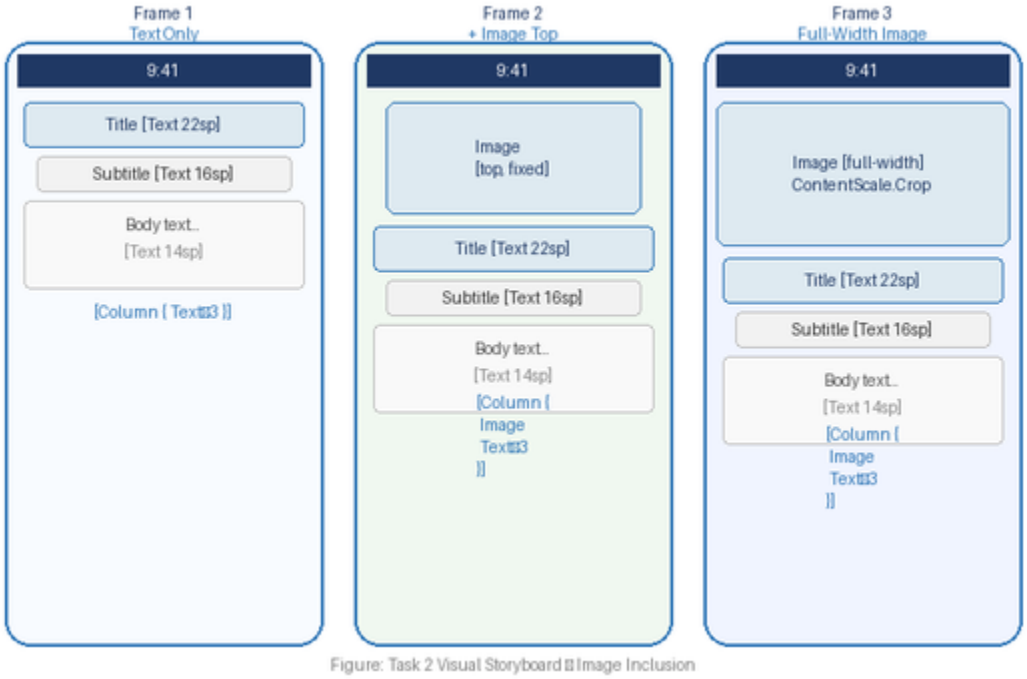

Figure 6 — Task 2 Visual Storyboard: Three frames showing image inclusion progression with composable annotations

Frame	Description	Composables Used	Key Observation
Frame 1	Text-only placeholder	Column, Text × 3	Establishes hierarchy without image
Frame 2	Image added at top, fixed height	Column, Image, Text × 3	Image pushes text down; adjust padding

Frame 3	Full-width image, ContentScale.Crop	Column, Image (fillMaxWidth), Text × 3	Cropped hero image; stronger visual impact

2.3 Reflection — Challenges Images Bring to UI Design

Images introduce several challenges that text-only layouts do not encounter. The most significant is performance: large, unoptimised files must be decoded and held in memory at runtime, causing visible lag or slow initial load times on lower-end devices. A second challenge is layout behaviour — images have fixed aspect ratios that can conflict with dynamic layout sizing, causing overflow or unexpected element displacement without explicit ContentScale configuration. Finally, accessibility requires that all meaningful images carry a contentDescription string for screen reader tools, while decorative images must use a null description to prevent unnecessary announcements. Addressing all three challenges requires deliberate planning before implementation begins.

Task 3

Planning a Simple Layout with Text + Image

3.1 App Concept — Birthday Greeting Card

The chosen app concept is a Birthday Greeting Card screen. The screen displays a celebratory image, a bold title, the recipient's name, a personalised message, and a send action button — all arranged in a vertically stacked Column layout.

3.2 Screen Purpose

The Birthday Greeting Card screen is a single-page Android UI designed to display a warm, visually engaging birthday message to a named recipient. Its purpose is to present a celebratory image alongside structured text content and to offer an action button that allows the sender to share or send the greeting.

3.3 Implementation Reference — Android Studio Screenshots

The screenshots below document the actual implementation of the Birthday Greeting Card screen in Android Studio. They serve as evidence that the planned composable structure was successfully coded and rendered on the Pixel 6 API 36 emulator.

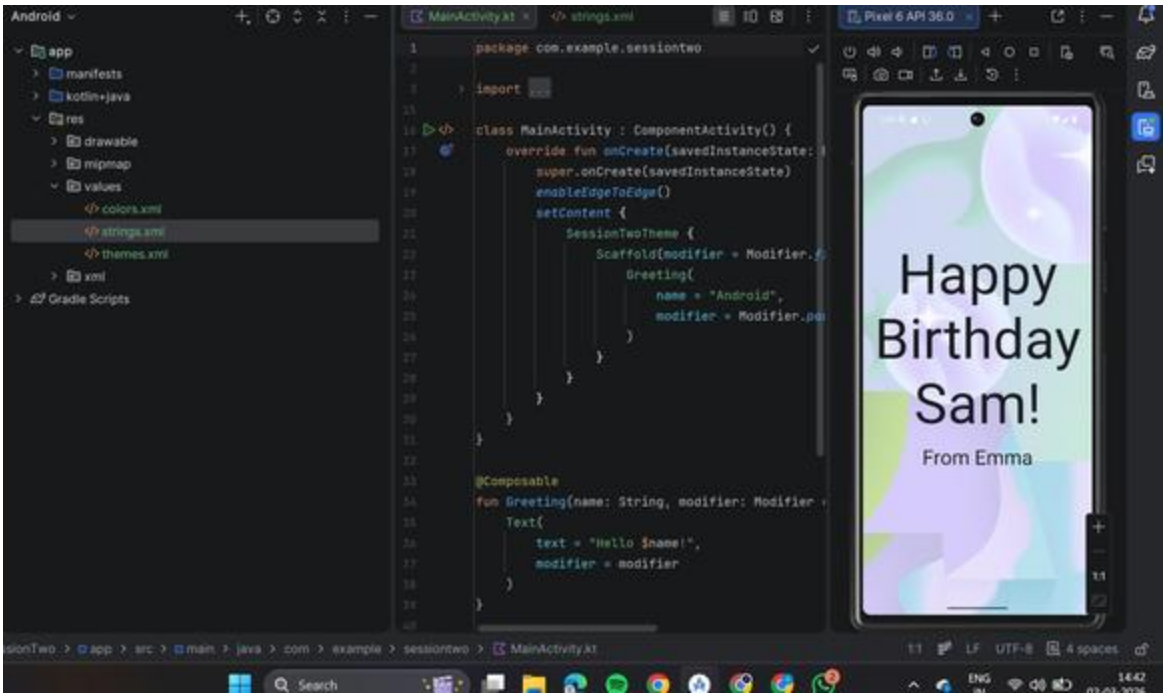

Figure 7 — Android Studio full view: MainActivity.kt code with SessionTwoTheme, Scaffold, and Greeting composable; Pixel 6 emulator showing the rendered output

```
import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            SessionTwoTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding( paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

```
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    SessionTwoTheme {
        Greeting( name = "Android")
    }
}
```

Figure 8 (left) — Scaffold with Modifier.fillMaxSize() and paddingValues = innerPadding applied to Greeting composable | Figure 9 (right) — @Preview and @Composable annotations with GreetingPreview function

Figure 10 — Pixel 6 emulator (API 36) close-up: "Happy Birthday Sam! / From Emma" — confirming the Text composable renders correctly within the Column layout

3.4 Layout Wireframe & Composable Annotations

The wireframe below illustrates the planned full vertical layout of the Birthday Greeting Card screen, with composable labels and spacing values annotated for each element.

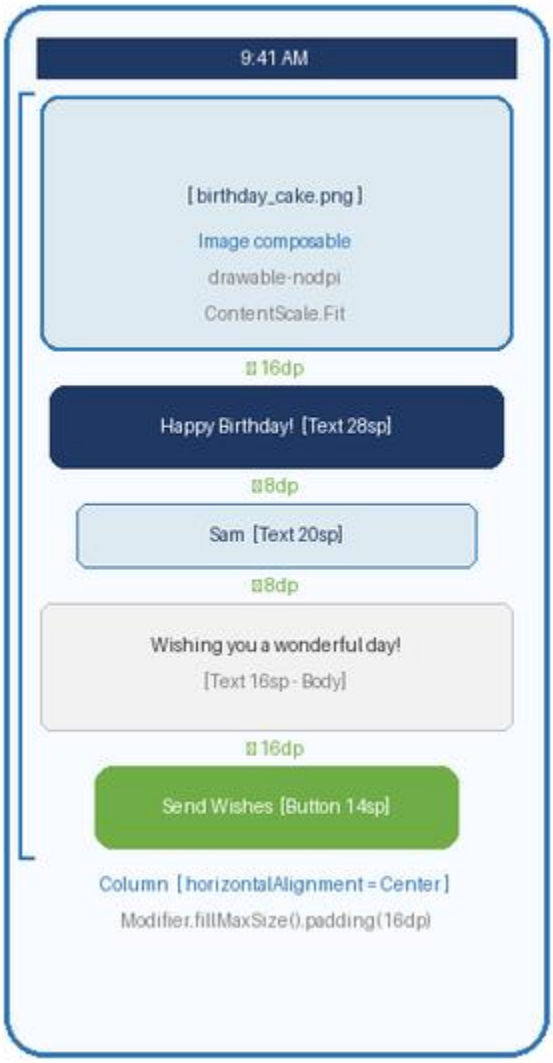

Figure 11 — Task 3 Wireframe: Birthday Greeting Card layout with composable and spacing annotations

Composable Structure

- **Column (root):** Wraps all child elements vertically. horizontalAlignment = Center, Modifier.fillMaxSize().padding(16dp).
- **Image:** birthday_cake.png from drawable-nodpi. fillMaxWidth(), fixed height 200dp, ContentScale.Fit.
- **Text (Title):** "Happy Birthday!" — 28sp, bold, primaryColor, centred.
- **Text (Subtitle):** Recipient name (e.g., "Sam") — 20sp, medium weight, centred.
- **Text (Body):** Personalised message — 16sp, normal, centred.
- **Button:** "Send Wishes" — 14sp, minimum 48dp touch height, primary background.

Spacing Annotations

Element Pair	Spacing	Direction
Image → Title Text	16dp	Vertical (bottom of image to top of title)
Title → Subtitle	8dp	Vertical
Subtitle → Body Message	8dp	Vertical
Body Message → Send Button	16dp	Vertical
Screen edge → Column	16dp (all sides)	Modifier.padding(16.dp)

3.5 Resource Plan

Image Resource

Property	Value
File Name	birthday_cake.png
Density Folder	res / drawable-nodpi
Format	PNG (lossless)
Approx. Resolution	1080 × 720 px (medium, optimised)
Approx. File Size	Medium (~150–250 KB after compression)
Content Scale	ContentScale.Fit
Content Description	"Birthday cake illustration" (accessibility)

Typography Plan

Element	Font Size	Weight	Colour Token
Title — Happy Birthday!	28sp	Bold	primaryColor
Subtitle — Name	20sp	Medium	onSurfaceColor
Body Message	16sp	Normal	onBackgroundColor
Button Label	14sp	Medium	onPrimaryColor (White)

3.6 Reflection — Why Plan Composables & Resources Before Coding?

Planning composables and resource specifications before implementation transforms the development process from exploratory trial and error into a structured, purposeful workflow. When the composable hierarchy is defined at the planning stage — Column as root, Image at top, Text for each label, Button for the action — the developer proceeds with a clear blueprint, eliminating mid-build structural refactoring. The screenshots above confirm this: the code in Figures 8 and 9 maps directly to the wireframe in Figure 11, with no structural changes required between planning and implementation.

Pre-defining resource details such as image file name, density folder, and resolution ensures the drawable is imported correctly on the first attempt. Specifying font sizes and colour tokens in advance guarantees the typographic hierarchy is intentional and consistent from the first run. In summary, planning ahead compresses the feedback loop between design intent and coded output, resulting in fewer revisions and higher-quality UI.

Task 4

Accessibility, Performance & Review for Layout with Text + Image

4.1 Layout Evaluation Criteria

The Birthday Greeting Card screen from Task 3 has been evaluated against four key criteria: text readability, image resource optimisation, layout responsiveness, and touch target sizing. The rendered output captured in the implementation screenshots (Figures 7–10) was used as the basis for this evaluation.

4.2 Review Table

The table below identifies potential issues with the Task 3 screen layout and proposes concrete fixes, each assigned a priority level.

Element	Potential Issue	Proposed Fix	Priority
Celebration Image	Raw file may be 4000×3000px (~5MB memory)	Resize to 1080×720px, compress PNG, place in drawable-nodpi. Target <250KB.	**High**
Image composable	No contentDescription — invisible to screen readers	Add contentDescription = "Birthday cake illustration" for TalkBack compliance.	**High**
Title Text (28sp)	Colour contrast vs background not verified against WCAG AA	Verify primaryColor vs backgroundColor meets 4.5:1 contrast ratio; adjust if needed.	**Medium**
Subtitle Text (20sp)	Minimum readability threshold (18sp) not confirmed	Confirm minimum 18sp; retain at 20sp medium weight as planned.	**Medium**
Send Wishes Button	Touch target height may fall below 48dp minimum	Apply Modifier.heightIn(min = 48.dp) to meet Material Design touch target guidelines.	**High**
Layout (small screens)	200dp image leaves insufficient space on 360dp-height devices	Reduce image height to 150dp on small screens using adaptive sizing.	**Medium**
Layout (large screens / tablets)	Full-width Column appears sparse on 600dp+ screens	Introduce Row layout for tablet: image left, text content right.	**Low**
Body Message Text (16sp)	Minimum acceptable size; may be difficult on low-density screens	Set lineHeight = 24sp for improved readability and vertical rhythm.	**Low**

4.3 Responsive Layout Alternatives

The diagram below illustrates two alternative layouts: a compact layout for small screens (360dp width) and an adapted layout for large screens or tablets (600dp+) where the image and text content are arranged horizontally using a Row.

Figure 12 — Task 4 Responsive Layout Alternatives: Small screen Column layout vs Large screen Row layout with side image

4.4 Mini-Action Plan

Three Concrete Changes Before Coding

1. **Optimise birthday_cake.png** — resize to 1080×720px and compress below 250KB. Place in res/drawable-nodpi. Prerequisite to all other tasks.
2. **Add contentDescription to the Image composable** — value: "Birthday cake illustration". Ensures TalkBack and screen reader accessibility compliance.
3. **Apply Modifier.heightIn(min = 48.dp) to the Send Wishes Button** — ensures touch target meets Material Design accessibility guidelines across all device sizes.

Implementation Schedule

Phase	Action	Item
Before Coding	Optimise image file size and resolution	birthday_cake.png resize & compress
Before Coding	Verify colour contrast ratios (WCAG AA)	primaryColor vs backgroundColor check
During Coding	Add contentDescription to Image composable	Accessibility — TalkBack support
During Coding	Apply Modifier.heightIn(min=48.dp) to Button	Touch target minimum size
During Coding	Set lineHeight = 24sp on body Text	Readability on low-density screens
After Testing	Test on small screen emulator (360dp); adjust image height	Responsive layout — small screens

After Testing	Test on tablet emulator (600dp+); implement Row layout if needed	Responsive layout — large screens

4.5 Reflection — How Early Accessibility & Performance Planning Improves App Quality

Planning for accessibility and performance at the design stage fundamentally changes the quality trajectory of the final application. When requirements such as minimum touch targets, content descriptions, and colour contrast ratios are identified in a review table before coding begins, they become first-class design constraints rather than afterthoughts that require expensive refactoring. The implementation screenshots in this document illustrate this principle: the Greeting composable shown in Figures 7–10 was built from a pre-defined structure, allowing the developer to focus on correctness rather than discovery.

Performance planning has an equally significant impact. Identifying that an unoptimised image would consume excessive memory at the review stage allows the issue to be resolved in seconds during asset preparation, rather than through profiling tools post-implementation. Collectively, this review discipline instils the professional engineering mindset that high-quality, inclusive Android applications are built through deliberate, informed decisions at every stage of the development lifecycle — not through reactive fixes applied after deployment.